

Paradigm ước lượng back-end (Back-End Estimation Paradigms)

PHỤ THUỘC A/04-uoc-luong-map-va-bundle-adjustment (MAP, Schur, marginalization — chương này là các lựa chọn khác nhau trong khung đó); A/05-observability-gauge-va-suy-bien (nhất quán và FEJ); B/08-mo-hinh-phan-du (structureless là kỹ thuật nền cho MSCKF). Phần quán tính tương ứng ở ../IMU_Fusion/ chương 8 và 9.

TÓM TẮT

Người mới học SLAM gặp một danh sách các kiến trúc back-end trông không liên quan: EKF, MSCKF, cửa sổ trượt, fixed-lag, iSAM2, full batch BA, pose graph. Mỗi cái có bài báo riêng, ký hiệu riêng, và cộng đồng riêng. Chương này lập luận rằng chúng là *một* bài toán MAP của A/04 với các lựa chọn khác nhau trên đúng hai trục: *marginalize những gì*, và *tuyến tính hoá lại bao nhiêu lần*. Hiểu hai trục đó thì danh sách sụp thành một bảng, và việc chuyển qua lại giữa các thể giới trở nên tự nhiên. Nội dung có bốn phần. Một: chứng minh sự tương đương, và chỉ ra chính xác chỗ bộ lọc đánh đổi cái gì. Hai: bốn kiến trúc chính đặt trên hai trục ấy. Ba: pose graph như một trường hợp riêng quan trọng — nó là một bài toán MAP mà mọi landmark đã bị marginalize. Bốn: hai hướng phát triển đáng biết là dạng căn bậc hai (ổn định số học) và certifiable (bảo đảm nghiệm toàn cục). Nếu chỉ nhớ một điều: *bộ lọc là bài toán tối ưu đã marginalize toàn bộ quá khứ và chỉ tuyến tính hoá một lần* — mọi khác biệt về chi phí và về tính nhất quán đều là hệ quả của câu đó.

KIẾN THỨC CẦN CÓ TRƯỚC

- **bộ lọc (filter)** — bộ ước lượng chỉ giữ trạng thái hiện tại và hiệp phương sai của nó, cập nhật tuần tự khi có phép đo mới. Mọi thông tin quá khứ được nén vào hai đại lượng đó.
- **smoothing** — bộ ước lượng giữ *nhiều* trạng thái quá khứ và giải lại cho tất cả khi có phép đo mới. Kết quả tốt hơn vì một phép đo muộn có thể sửa một ước lượng sớm.
- **cửa sổ trượt (sliding window)** — smoothing trên một cửa sổ kích thước cố định; trạng thái ra khỏi cửa sổ được marginalize thành nhân tử prior.
- **fixed-lag smoothing** — tên khác của cửa sổ trượt, nhấn mạnh rằng ước lượng cho một thời điểm được chốt sau một độ trễ cố định.
- **smoothing gia tăng (incremental smoothing)** — giữ *toàn bộ* lịch sử nhưng chỉ tính lại phần bị ảnh hưởng khi có phép đo mới. iSAM2 làm điều này qua cấu trúc Bayes tree.
- **pose graph** — đồ thị nhân tử chỉ gồm các tư thế, với các cạnh là ràng buộc tư thế tương đối. Landmark đã bị khử; đây là dạng nén nhất của bài toán còn giữ được tính nhất quán toàn cục.
- **dạng căn bậc hai (square-root form)** — làm việc với thừa số R sao cho $\Lambda = R^T R$ thay vì với Λ . Số điều kiện giảm còn căn bậc hai, nên ổn định số học tốt hơn hẳn.
- **certifiable** — thuật toán không chỉ trả về nghiệm mà còn trả về *chứng chỉ* rằng nghiệm đó là cực tiểu toàn cục — điều mà Gauss-Newton không bao giờ cho được.

CÂU HỎI MỞ ĐẦU

- Q1. EKF-SLAM và bundle adjustment cùng giải một bài toán MAP. Vậy khác biệt thật nằm ở đâu, và nêu chính xác hai lựa chọn phân biệt chúng.

- Q2. EKF-SLAM có chi phí bậc hai theo số landmark mỗi bước. Vì sao lại là bậc hai, khi mà trận thông tin của bài toán thì thưa?
- Q3. MSCKF giữ một cửa sổ tư thế và khử landmark. Nó rẻ và chính xác. Vì sao nó *không* phải một hệ SLAM, và điều đó có hậu quả gì cho sản phẩm?
- Q4. iSAM2 giữ toàn bộ lịch sử nhưng chạy thời gian thực. Nghe như nó có tất cả mà không mất gì. Điều gì nó vẫn phải trả giá, và khi nào cái giá đó trở nên nghiêm trọng?
- Q5. Dạng căn bậc hai cho cùng nghiệm với cùng chi phí bậc độ lớn, nhưng ổn định số học tốt hơn hẳn. “Tốt hơn hẳn” ở đây là bao nhiêu, và nó đến từ đâu?

1.1 Đặt vấn đề

Ai học SLAM theo thứ tự lịch sử sẽ gặp một danh sách gây bối rối. Đầu tiên là EKF-SLAM, thống trị những năm 1990 và 2000. Rồi PTAM [1] chứng minh rằng bundle adjustment chạy được thời gian thực, và cộng đồng chuyển sang tối ưu hoá. Rồi MSCKF [2] cho thấy bộ lọc vẫn cạnh tranh nếu làm đúng. Rồi iSAM2 [3] cho thấy smoothing đầy đủ cũng chạy thời gian thực được. Bốn câu chuyện, bốn cộng đồng, bốn bộ ký hiệu.

Cách học đó để lại một ấn tượng sai: rằng đây là những phương pháp *cạnh tranh* và ta phải chọn một. Ấn tượng ấy làm người học phải học lại từ đầu mỗi lần đổi hệ thống, và tệ hơn, nó che mất việc các đánh đổi là gì.

Bài toán gốc của chương: *tồn tại một khung nào mà trong đó cả bốn đều là trường hợp riêng không?* Câu trả lời là có, và khung đó chính là bài toán MAP ở A/04. Mọi kiến trúc trong danh sách trên đều cực tiểu hoá cùng một hàm mục tiêu; chúng khác nhau ở hai lựa chọn kỹ thuật:

Trục một — *marginalize* những gì. Giữ lại bao nhiêu quá khứ trong bài toán? Từ “không giữ gì” (bộ lọc) tới “giữ tất cả” (full batch).

Trục hai — *tuyến tính hoá lại bao nhiêu lần*. Jacobian được tính tại điểm nào, và có tính lại khi ước lượng di chuyển không? Từ “một lần, không bao giờ sửa” (EKF) tới “mỗi vòng lặp, cho mọi biến” (full batch).

Hai trục ấy giải thích *tất cả* khác biệt về chi phí, độ chính xác và tính nhất quán. Và quan trọng hơn, chúng cho một cách suy nghĩ: khi thiết kế hệ thống, bạn không chọn giữa bốn cái tên mà chọn một điểm trên một mặt phẳng hai chiều.

Ai gặp bài toán này? Mọi người thiết kế back-end, và quyết định này định hình yêu cầu tính toán, độ trễ, và khả năng đóng vòng. Nó cũng là quyết định khó đảo ngược nhất trong cả hệ thống — đổi detector thì mất một tuần, đổi kiến trúc back-end thì viết lại gần hết.

TRỰC GIÁC

Hình dung bạn đang ghi chép một chuyến đi dài.

Cách bộ lọc: bạn chỉ giữ *một* trang giấy, ghi “tôi đang ở đây, và tôi chắc chắn chừng này”. Mỗi khi có thông tin mới, bạn cập nhật trang đó rồi *xoá* thông tin cũ — vì nó đã được gộp vào con số hiện tại. Rất rẻ, bộ nhớ không đổi. Cái giá: nếu ba giờ sau bạn phát hiện một suy luận lúc sáng là sai, bạn không sửa được. Trang giấy không còn ghi suy luận đó nữa; nó chỉ còn kết quả.

Cách full batch: bạn giữ *toàn bộ* nhật ký, mọi quan sát từ đầu chuyến đi. Mỗi khi có thông tin mới, bạn đọc lại tất cả và suy luận lại từ đầu. Kết quả tốt nhất có thể, vì một thông tin muộn có thể sửa một hiểu lầm sớm. Cái giá: chi phí tăng theo độ dài chuyến đi, nên tới ngày thứ mười thì không kịp nữa.

Cách của sổ trượt: bạn giữ nhật ký của *ba ngày gần nhất* và tóm tắt mọi thứ cũ hơn thành một đoạn ghi chú. Chi phí không đổi, và bạn vẫn sửa được các hiểu lầm trong ba ngày qua. Cái giá nằm ở đoạn tóm tắt: một khi đã tóm tắt, bạn không mở lại được, và nếu lúc tóm tắt bạn đang hiểu sai thì cái hiểu sai ấy bị đóng băng.

Cách smoothing gia tăng: bạn giữ toàn bộ nhật ký nhưng tổ chức nó *rất khéo*, sao cho khi có thông tin mới bạn chỉ phải đọc lại đúng những trang bị ảnh hưởng. Nếu thông tin mới chỉ liên quan tới hôm nay thì bạn chỉ đọc hôm nay. Nhưng nếu bạn nhận ra hôm nay mình đang ở đúng chỗ đã đi qua tuần trước — một loop closure — thì bạn phải đọc lại cả tuần, và lúc đó chi phí vọt lên.

Bốn cách, một bài toán. Khác biệt duy nhất: giữ lại bao nhiêu, và có cho phép sửa lại suy luận cũ hay không.

1.2 Bộ lọc và tối ưu là cùng một bài toán

A. Chứng minh sự tương đương

Xét bài toán MAP với các trạng thái $\mathbf{x}_{0:k}$ và phép đo tới thời điểm k . Full batch giải cho tất cả:

$$\hat{\mathbf{x}}_{0:k} = \arg \min \left[\underbrace{\|\mathbf{x}_0 - \bar{\mathbf{x}}_0\|_{\Sigma_0}^2}_{\text{prior}} + \sum_i \|\mathbf{r}_i^{\text{do}}\|_{\Omega_i}^2 + \sum_i \|\mathbf{r}_i^{\text{chuyển}}\|_{Q_i}^2 \right].$$

Bây giờ áp dụng marginalization của A/04 mục 5 một cách cực đoan: sau mỗi bước thời gian, marginalize *mọi* trạng thái trừ trạng thái mới nhất. Theo mục 5A ở chương đó, marginalize một khối biến Gauss chính là phần bù Schur, và kết quả là một nhân tử prior trên các biến còn lại. Vì chỉ còn một trạng thái, prior ấy là một Gauss đơn giản: một trung bình và một hiệp phương sai.

Đó *chính xác* là những gì EKF giữ. Bước dự đoán của EKF là việc lan truyền prior qua nhân tử chuyển động; bước cập nhật là việc gộp nhân tử đo mới vào prior. Cùng công thức, cùng kết quả.

Nhưng có một khác biệt thứ hai, và nó là khác biệt quan trọng hơn. Trong full batch, khi ta thêm phép đo mới, ta *tuyến tính hoá lại* mọi phần dư tại ước lượng mới nhất. Trong EKF, Jacobian của một phép đo được tính *một lần* tại ước lượng lúc đó và không bao giờ sửa. Khi ước lượng di chuyển — và nó luôn di chuyển — Jacobian cũ trở nên lỗi thời, nhưng thông tin đã được nén vào hiệp phương sai và không tách ra được nữa.

CẦN NHỚ

Bộ lọc = bài toán tối ưu đã marginalize toàn bộ quá khứ, và chỉ tuyến tính hoá một lần.

Hai vế của câu này giải thích mọi thứ còn lại.

Về *marginalize toàn bộ quá khứ* giải thích chi phí: trạng thái không lớn lên theo thời gian, nên bộ lọc chạy được vô hạn. Nó cũng giải thích một hệ quả ít được nói: bộ lọc *không thể* sửa lại một ước lượng cũ, vì ước lượng cũ không còn tồn tại như một biến.

Về *tuyến tính hoá một lần* giải thích độ chính xác và tính nhất quán. Với mô hình phi tuyến mạnh — và phép chiếu camera phi tuyến mạnh — Jacobian tính tại một ước lượng tệ sẽ sai, và cái sai ấy bị đóng băng vĩnh viễn. Tệ hơn, như A/05 mục 6 chỉ ra, các Jacobian tính tại các điểm khác nhau *không tương thích* và tạo ra thông tin giả theo hướng không quan sát được. Đó là gốc của bệnh quá tự tin, và là lý do FEJ tồn tại.

Hệ quả thiết kế: nếu bạn cần chi phí không đổi và chấp nhận không đóng vòng, dùng bộ lọc và **bắt buộc** phải xử lý nhất quán (FEJ hoặc InEKF). Nếu bạn cần đóng vòng và sửa lại quá khứ, bạn cần giữ quá khứ, tức smoothing ở một dạng nào đó.

B. Vì sao EKF-SLAM có chi phí bậc hai

Đây là câu trả lời cho câu hỏi mở đầu thứ hai, và nó là một nghịch lý đáng hiểu.

Trong bài toán MAP, ma trận *thông tin* Λ của SLAM *thừa* — đó là toàn bộ nội dung của A/04 mục 4B. Nhưng EKF không giữ Λ ; nó giữ $\Sigma = \Lambda^{-1}$. Và như A/06 mục 2B đã nêu, *nghịch đảo của một ma trận thừa nói chung là đặc*.

Lý do trực giác: hai landmark chưa bao giờ được nhìn thấy cùng lúc vẫn *tương quan* với nhau, vì cả hai đều tương quan với quỹ đạo nối chúng. Thông tin “chúng không nối trực tiếp” nằm trong Λ ; thông tin “chúng vẫn tương quan” nằm trong Σ . Bộ lọc chọn biểu diễn thứ hai, và phải trả giá.

Hậu quả: với n landmark, Σ có $O(n^2)$ phần tử, và mỗi bước cập nhật EKF phải chạm vào tất cả — chi phí $O(n^2)$ mỗi bước, $O(n^2)$ bộ nhớ. Với vài nghìn landmark là hết. Đây là lý do kỹ thuật khiến EKF-SLAM không mở rộng được, và là lý do MSCKF quan trọng: bằng cách khử landmark qua structureless (B/08 mục 3C), nó giữ trạng thái chỉ gồm một cửa sổ tư thế, và chi phí không còn phụ thuộc số landmark.

Có một họ thứ ba đáng biết: *information filter*, giữ Λ thay vì Σ . Nó giữ được tính thừa — nhưng để lấy ra ước lượng thì phải giải một hệ, và để lấy hiệp phương sai thì phải nghịch đảo. Nói cách khác, nó chuyển chi phí từ lúc cập nhật sang lúc truy vấn. Với SLAM, nơi ta cập nhật liên tục nhưng chỉ thỉnh thoảng cần hiệp phương sai đầy đủ, đây thường là đánh đổi tốt hơn — và đó chính là cách mà các phương pháp smoothing hiện đại làm việc.

I 1.3 Bốn kiến trúc trên hai trục

Bảng 1.1. Bốn kiến trúc đặt trên hai trục: giữ lại bao nhiêu quá khứ, và tuyến tính hoá lại bao nhiêu. Mọi khác biệt về chi phí và nhất quán đều là hệ quả của hai cột đầu.

Kiến trúc	Giữ lại	Tuyến tính hoá lại	Hệ quả	Đại diện
Bộ lọc (EKF)	Chỉ trạng thái hiện tại	Không bao giờ	Chi phí không đổi theo thời gian; bắt nhất nếu không FEJ; không sửa được quá khứ	EKF-SLAM, ESKF
Bộ lọc cửa sổ (MSCKF)	Cửa sổ tư thế, không có landmark	Không bao giờ	Chi phí không phụ thuộc số landmark; không có bản đồ nên không đóng vòng	MSCKF, OpenVINS
Cửa sổ trượt	Cửa sổ tư thế + landmark; cũ hơn thì marginalize	Trong cửa sổ, mỗi vòng lặp	Chi phí không đổi; sửa được quá khứ gần; prior bị đóng băng gây fill-in và bắt nhất	VINS-Mono, OKVIS, Basalt
Smoothing gia tăng	Toàn bộ lịch sử	Phần bị ảnh hưởng, mỗi lần	Chính xác như full batch; chi phí thấp khi cập nhật cục bộ, vọt lên khi đóng vòng	iSAM2, GTSAM
Full batch	Toàn bộ	Tất cả, mỗi vòng lặp	Nghiem tốt nhất có thể; không chạy thời gian thực	BA toàn cục, COLMAP

Ba nhận xét đọc ra từ bảng, và chúng quan trọng hơn bản thân bảng.

Thứ nhất, hai cột đầu là nguyên nhân, cột thứ ba là kết quả. Không có gì trong cột “hệ quả” là một lựa chọn thiết kế độc lập — tất cả đều suy ra từ hai lựa chọn trước. Điều này có nghĩa là nếu bạn muốn một hệ có tính chất nào đó ở cột ba, bạn phải tìm nó ở hai cột đầu.

Thứ hai, MSCKF không phải SLAM. Đây là câu trả lời cho câu hỏi mở đầu thứ ba. Vì landmark bị khử ngay sau khi dùng, không có bản đồ nào tồn tại, nên không có gì để nhận ra lại khi quay về chỗ cũ. MSCKF là một hệ *odometry* xuất sắc: trôi chậm, rẻ, ổn định. Nhưng trôi của nó là đơn điệu — nó không bao giờ tự sửa. Với sản phẩm chạy trong một không gian hữu hạn và lặp lại — robot lau nhà, AGV kho hàng — điều đó không chấp nhận được, và ta cần một lớp SLAM đầy đủ ở trên. Cách phổ biến trong thực tế là ghép: MSCKF hoặc cửa sổ trượt cho odometry tần số cao, cộng một pose graph cho nhất quán toàn cục (B/11).

Thứ ba, iSAM2 vẫn phải trả giá. Đây là câu trả lời cho câu hỏi mở đầu thứ tư. Nó giữ toàn bộ lịch sử và chỉ tính lại phần bị ảnh hưởng, nhờ Bayes tree tổ chức biến theo một thứ tự khử tốt [3]. Khi cập nhật là cục bộ theo thời gian — thêm một khung, thêm vài landmark — phần bị ảnh hưởng nhỏ và chi phí thấp. Nhưng một *loop closure* nối một tư thế mới với một tư thế rất cũ, và nó làm mất hiệu lực một phần lớn của cây; chi phí vọt lên. Với bản đồ lớn và loop closure thường xuyên, cái giá đó trở nên nghiêm trọng, và đó là lý do các hệ thực tế thường tách loop closure ra một luồng riêng chạy chậm hơn (C/13).

1.4 Pose graph: bài toán MAP đã nén

Pose graph là một trường hợp riêng quan trọng tới mức đáng một mục riêng.

Ý tưởng: khử *mọi* landmark, giữ chỉ tư thế. Mỗi cạnh của đồ thị là một ràng buộc tư thế tương đối $\hat{T}_{ij}$ kèm hiệp phương sai, và phần dư là

$$\mathbf{r}_{ij} = \log(\hat{T}_{ij}^{-1} T_i^{-1} T_j)^\vee.$$

Các ràng buộc ấy đến từ hai nguồn: odometry (giữa hai tư thế liên tiếp) và loop closure (giữa hai tư thế xa nhau về thời gian).

Ba lý do pose graph quan trọng.

Một — kích thước. Với m tư thế và n landmark, bài toán đầy đủ có $6m + 3n$ ẩn với $n \gg m$; pose graph chỉ có $6m$. Điều này biến một bài toán không chạy nổi thành một bài toán giải được trong vài giây, kể cả với hàng chục nghìn tư thế.

Hai — tách mỗi quan tâm. Việc sửa trôi tích lũy là bài toán *toàn cục* và không cần độ chính xác cao ở quy mô nhỏ; việc tinh chỉnh cấu trúc cục bộ là bài toán *địa phương*. Tách chúng ra cho phép chạy hai luồng ở hai tần số khác nhau — một kiến trúc mà mọi hệ SLAM thật đều dùng.

Ba — nền cho certifiable. Vì pose graph chỉ chứa tư thế, nó có cấu trúc đại số đặc biệt cho phép các phương pháp bảo đảm nghiệm toàn cục (mục 5B).

Số bậc tự do của pose graph phải khớp với cấu hình cảm biến, đúng như A/05 mục 3B: $Sim(3)$ cho mono [4], $SE(3)$ cho stereo, và bốn bậc cho VIO vì trọng lực ghim roll với pitch. Đây không phải tối ưu hoá mà là tính đúng đắn: tối ưu một bậc tự do mà dữ liệu đã xác định chỉ tạo thêm đường cho sai số tích lũy.

Cái giá của pose graph, và nó đáng nói rõ: *thông tin bị nén một cách không phục hồi được*. Một cạnh với hiệp phương sai 6×6 không mô tả đầy đủ những gì hàng trăm quan sát landmark đã nói. Cụ thể, các cạnh khác nhau *tương quan* với nhau (vì chúng chia nhau các landmark), và pose graph chuẩn coi chúng độc lập — đúng nguồn số một của bệnh quá tự tin ở A/06 mục 4B. Đây là một xấp xỉ được chấp nhận rộng rãi vì lợi ích tính toán quá lớn, nhưng nên biết rằng nó là một xấp xỉ.

1.5 Hai hướng phát triển đáng biết

A. Dạng căn bậc hai

Đây là câu trả lời cho câu hỏi mở đầu thứ năm, và nó là một trong những cải tiến có tỉ lệ lợi ích trên chi phí cao nhất trong cả cây.

Thay vì dựng và phân tích $\Lambda = \sum H^\top \Omega H$, làm việc trực tiếp với một thừa số R sao cho $\Lambda = R^\top R$, thu được bằng phân tích QR của ma trận Jacobian đã whitened.

Lợi ích số học định lượng được, và đó là điểm mạnh của lập luận. Số điều kiện của Λ là *bình phương* số điều kiện của R :

$$\text{cond}(\Lambda) = \text{cond}(R^\top R) = \text{cond}(R)^2.$$

Nghĩa là việc dựng Λ tương minh *phá huỷ một nửa số chữ số có nghĩa*. Với bài toán có số điều kiện 10^8 — hoàn toàn bình thường trong BA có điểm ở nhiều thang độ sâu khác nhau — ta mất tám chữ số, và số học độ chính xác kép chỉ có mười sáu. Làm việc với R giữ được tám chữ số đó. (Cửa kiểm chứng a: suy dẫn trực tiếp từ định nghĩa số điều kiện qua giá trị kỳ dị.)

Lợi ích này lớn nhất đúng ở chỗ bài toán gần suy biến — tức đúng các cấu hình ở A/05. Basalt [5] và $\sqrt{\text{BA}}$ [6] khai thác điều này, và $\sqrt{\text{BA}}$ còn cho thấy nó cho phép dùng số học *độ chính xác đơn* trong nhiều trường hợp — tức nhanh gấp đôi và tốn nửa bộ nhớ, một lợi ích thật cho hệ nhúng (D/20).

Cái giá: phân tích QR đắt hơn Cholesky khoảng hai lần cho cùng bài toán, và cài đặt phức tạp hơn, đặc biệt khi kết hợp với phần bù Schur. Đây là loại đánh đổi mà một hệ sản phẩm nên cân nhắc nghiêm túc, vì tính ổn định thường quan trọng hơn tốc độ.

B. Certifiable: bảo đảm nghiệm toàn cục

Gauss-Newton và LM chỉ tìm được cực tiểu *địa phương*, và chúng không bao giờ cho biết nghiệm tìm được có phải toàn cục hay không. Với pose graph có loop closure, hàm mục tiêu *rất* không lồi và cực tiểu địa phương xấu tồn tại thật — một bản đồ “gập” vào chính nó là một cực tiểu địa phương hoàn toàn hợp lệ về mặt toán học.

SE-Sync [7] khai thác một cấu trúc đặc biệt: bài toán pose graph có thể viết lại thành một bài toán tối ưu trên đa tạp các ma trận bán xác định, và trong điều kiện nhiễu vừa phải, một nơi lỏng lẻo của nó có nghiệm *trùng* với nghiệm của bài toán gốc. Khi điều kiện thoả, thuật toán trả về nghiệm kèm một *chứng chỉ* rằng đó là cực tiểu toàn cục.

Điều kiện “nhiều vừa phải” là chỗ đáng chú ý và cũng là giới hạn: với nhiễu lớn hoặc với outlier, bảo đảm mất hiệu lực. Nên certifiable không thay thế được back-end bền ở B/11 — hai thứ giải hai bài toán khác nhau, một cái lo cực tiểu địa phương và một cái lo dữ liệu sai.

Một hướng liên quan và thực dụng hơn: **rotation averaging** [8]. Tách bài toán thành hai bước — giải cho mọi phép xoay trước (bài toán này dễ hơn nhiều và có nghiệm gần toàn cục), rồi giải cho tịnh tiến với xoay đã cố định (bài toán này gần tuyến tính). Đây là nền của SfM toàn cục, đối lập với SfM tuần tự của COLMAP, và nó cho thấy rằng việc *chọn thứ tự giải* có thể quan trọng ngang việc chọn bộ giải.

CẠM BẮY

Đừng chọn kiến trúc back-end theo độ chính xác công bố. Ba lý do.

Thứ nhất, các con số không so sánh được: chúng chạy trên các bộ dữ liệu khác nhau, với front-end khác nhau, và front-end thường ảnh hưởng tới ATE nhiều hơn back-end.

Thứ hai, và quan trọng hơn, *độ chính xác trung bình không phải tiêu chí quyết định*. Bốn tiêu chí sau thường quyết định hơn trong sản phẩm: (a) **chi phí xấu nhất** — một hệ nhanh trung bình nhưng thỉnh thoảng mất một giây là không dùng được trong vòng điều khiển;

(b) **có đóng vòng được không** — quyết định hệ có trôi vô hạn hay không, và đó là khác biệt giữa dùng được và không dùng được trong không gian lặp lại; (c) **tính nhất quán** — hiệp phương sai có đáng tin không (A/06); (d) **hành vi khi hỏng** — nó phân kỳ ồn ào hay trôi im lặng?

Thứ ba, quyết định này khó đảo ngược nhất trong cả hệ thống. Đổi detector mất một tuần; đổi kiến trúc back-end là viết lại gần hết. Nên nó xứng đáng được quyết định dựa trên yêu cầu sản phẩm, không dựa trên bảng xếp hạng.

Kiến thức này đang sống ở đâu (công trình đang dùng chính ý tưởng của chương; chốt 9/2026)

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
OpenVINS [9]	MSCKF với FEJ, hiệu chuẩn trực tuyến, mô phỏng có chân trị	Cài đặt bộ lọc sạch nhất; đọc để thấy cái giá của “tuyến tính hoá một lần”
VINS-Fusion [10]	Cửa sổ trượt + marginalization + pose graph riêng	Kiến trúc hai tầng điển hình: odometry nhanh + nhất quán toàn cục chậm
GTSAM [3]	iSAM2, Bayes tree, khử biến gia tăng	Cài đặt tham chiếu của smoothing gia tăng; đọc để hiểu vì sao loop closure đắt
Basalt [5]	Cửa sổ trượt dạng căn bậc hai; chất lượng kỹ thuật rất cao	Mục 5A trong một hệ thật; cũng là ví dụ tốt về kỹ thuật phần mềm
SE-Sync [7]	Pose graph certifiable với chứng chỉ toàn cục	Mục 5B; đọc để biết ranh giới của điều kiện “nhiều vừa phải”
ORB-SLAM3 [11]	Local BA + pose graph + BA toàn cục, ba luồng	Cho thấy các paradigm <i>kết hợp</i> chứ không loại trừ nhau

MINI PROJECT SLAM · Cài cùng một bài toán bằng ba kiến trúc, rồi đo cái mà bảng xếp hạng không đo · 4 ngày

Mục tiêu	tự thấy rằng ba kiến trúc cho cùng nghiệm khi mọi thứ thuận lợi, và thấy chúng phân kỳ ở <i>đâu</i> — vì chỗ phân kỳ mới là thứ quyết định lựa chọn.
Dữ liệu	tự sinh để có chân trị, cộng một chuỗi EuRoC để có dữ liệu thật.
Các bước	cài ba back-end trên cùng front-end và cùng dữ liệu: (a) EKF với toàn bộ landmark trong trạng thái, (b) cửa sổ trượt có marginalization, (c) full batch chạy lại từ đầu mỗi khung (chậm, làm chuẩn vàng); đo bốn thứ theo thời gian — ATE, thời gian mỗi bước (<i>cả trung bình lẫn phân vị 99</i>), bộ nhớ, và NEES so với chân trị; rồi bật/tắt FEJ cho (a) và (b) và đo lại NEES; cuối cùng thêm một loop closure và xem cái nào sửa được trôi.
Xong khi	bạn có bốn đường cong theo thời gian cho ba kiến trúc. Bạn phải quan sát được: (1) chi phí của (a) tăng bậc hai theo số landmark trong khi (b) không đổi; (2) NEES của (a) và (b) lệch lên khi không có FEJ và gần số chiều khi có — xác nhận A/05 mục 6; (3) chỉ (c) và (b) sửa được trôi khi có loop closure, còn (a) thì không nếu landmark đã ra khỏi trạng thái. Điểm quan trọng nhất: phân vị 99 của thời gian có thể kể một câu chuyện rất khác với trung bình.
Nối với	B/11-loop-closure-va-nhat-quan-toan-cuc xây trên pose graph ở mục 4; D/20-toi-uu-hieu-nang-va-he-thong lấy phân vị 99 làm tiêu chí thiết kế chứ không phải trung bình.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Chúng cực tiểu hoá *cùng một* hàm mục tiêu MAP. Hai lựa chọn phân biệt chúng: (1) *marginalize những gì* — EKF marginalize toàn bộ quá khứ sau mỗi bước, BA giữ tất cả; (2) *tuyến tính hoá lại bao nhiêu lần* — EKF tính Jacobian một lần tại ước lượng lúc đó và không bao giờ sửa, BA tính lại mọi Jacobian tại ước lượng tốt nhất hiện có ở mỗi vòng lặp. Về một giải thích chi phí (trạng thái không lớn lên) và giới hạn (không sửa được quá khứ). Về hai giải thích độ chính xác và tính nhất quán — Jacobian lỗi thời bị đóng băng, và các Jacobian tại các điểm khác nhau không tương thích nên tạo ra thông tin giả.
- A2. Vì EKF giữ $\Sigma = \Lambda^{-1}$ chứ không giữ Λ , và *ngịch đảo của một ma trận thưa nói chung là đặc*. Trực giác: hai landmark chưa bao giờ nhìn thấy cùng lúc vẫn tương quan, vì cả hai đều tương quan với quỹ đạo nổi chúng; Λ ghi “không nối trực tiếp”, còn Σ ghi “vẫn tương quan”. Với n landmark, Σ có $O(n^2)$ phần tử và mỗi cập nhật chạm tất cả. Có một lối thoát là information filter (giữ Λ , thưa) nhưng nó chuyển chi phí sang lúc truy vấn — và đó chính là cách các phương pháp smoothing hiện đại làm.
- A3. Vì landmark bị khử ngay sau khi dùng (structureless, B/08 mục 3C), nên *không có bản đồ nào tồn tại* — không có gì để nhận ra lại khi quay về chỗ cũ. MSCKF là odometry xuất sắc: trôi chậm, rẻ, ổn định, chi phí không phụ thuộc số landmark. Nhưng trôi của nó *đơn điệu* — nó không bao giờ tự sửa. Với sản phẩm chạy trong không gian hữu hạn và lặp lại (robot lau nhà, AGV kho), điều đó không chấp nhận được. Cách xử lý thực tế là ghép hai tầng: MSCKF hoặc cửa sổ trượt cho odometry tần số cao, cộng một pose graph cho nhất quán toàn cục.
- A4. Nó trả giá ở *loop closure*. Bayes tree cho phép chỉ tính lại phần bị ảnh hưởng, và khi cập nhật cục bộ theo thời gian — thêm một khung — phần đó nhỏ. Nhưng một loop closure nổi tư thế mới với tư thế rất cũ, làm mất hiệu lực một phần lớn cây, và chi phí vọt lên. Cái giá nghiêm trọng khi bản đồ lớn và loop closure thường xuyên — đúng tình huống của robot chạy lâu trong không gian lặp lại. Đó là lý do các hệ thực tế tách loop closure ra một luồng riêng chạy chậm hơn thay vì để nó chặn luồng tracking.
- A5. Tốt hơn *một cách định lượng được*: số điều kiện của R là *căn bậc hai* của số điều kiện của Λ , vì $\Lambda = R^T R$. Dựng Λ tường minh phá huỷ một nửa số chữ số có nghĩa. Với bài toán có $\text{cond} = 10^8$ — bình thường trong BA có điểm ở nhiều thang độ sâu — ta mất tám chữ số trong khi độ chính xác kép chỉ có mười sáu. Lợi ích lớn nhất đúng ở chỗ bài toán gần suy biến, tức các cấu hình ở A/05. Nó còn cho phép dùng độ chính xác đơn trong nhiều trường hợp [6], tức nhanh gấp đôi và tốn nửa bộ nhớ. Cái giá: QR đắt hơn Cholesky khoảng hai lần và cài đặt phức tạp hơn.

NÓI LẠI THẬT ĐƠN GIẢN

Bạn đang đi bộ đường dài và ghi lại mình đã đi qua đâu. Có bốn cách ghi, và chúng khác nhau đúng ở một điểm: bạn giữ lại bao nhiêu, và bạn có cho phép mình sửa lại những gì đã ghi hay không.

Cách một: chỉ giữ một mẩu giấy ghi “tôi đang ở đây”. Mỗi lần biết thêm điều gì, bạn sửa mẩu giấy rồi quên thông tin cũ đi. Rất nhẹ — mẩu giấy không dày lên dù bạn đi bao lâu. Nhưng chiều nay nếu bạn nhận ra rằng buổi sáng mình đã hiểu nhầm một ngã rẽ, bạn không sửa lại được, vì trên giấy không còn buổi sáng nữa.

Cách hai: giữ toàn bộ nhật ký từ đầu. Mỗi lần biết thêm điều gì, bạn đọc lại tất cả và nghĩ lại từ đầu. Kết quả luôn tốt nhất có thể. Nhưng đến ngày thứ mười, đọc lại hết mất cả buổi, và bạn không còn đi được nữa.

Cách ba: giữ nhật ký ba ngày gần nhất, còn cũ hơn thì tóm tắt thành một đoạn. Nhẹ, và vẫn sửa được những gì trong ba ngày. Cái bẫy nằm ở đoạn tóm tắt: một khi viết xong thì không mở lại được, và nếu lúc viết bạn đang hiểu nhầm thì cái hiểu nhầm đó đóng băng luôn ở đó.

Cách bốn: giữ hết, nhưng sắp xếp cực kỳ khéo — đánh dấu, phân mục — sao cho khi biết thêm điều gì, bạn chỉ phải đọc lại đúng những trang liên quan. Thường thì chỉ vài trang. Nhưng có một loại thông tin đặc biệt đắt: khi bạn nhận ra *chỗ mình đang đứng hôm nay chính là chỗ đã đi qua tuần trước*. Lúc đó mọi thứ ở giữa đều bị ảnh hưởng, và bạn phải đọc lại cả tuần.

Bốn cách, một việc. Và điều đáng nhớ nhất là: không có cách nào tốt hơn tuyệt đối. Nếu bạn chỉ cần biết mình đang ở đâu ngay bây giờ, cách một là đủ và rẻ nhất. Nếu bạn cần một tấm bản đồ đúng của cả chuyến đi, bạn buộc phải giữ lại quá khứ, và buộc phải trả giá cho nó.

Chỗ không rút gọn được: chỗ không rút gọn được là cái bẫy của đoạn tóm tắt ở cách ba. Ý tưởng “nén quá khứ lại nhưng giữ nguyên điều đã học được” nghe đơn giản, nhưng khi nén, các trang còn lại trở nên dính vào nhau theo những cách khó theo dõi, và điều đã học được thì ghi bằng một hệ quy chiếu đã lỗi thời. Đó là lý do các hệ cửa sổ trượt phức tạp hơn nhiều so với vẻ ngoài, và là lý do FEJ tồn tại.

Một câu: Bộ lọc là bài toán tối ưu đã quên hết quá khứ và chỉ được nghĩ một lần — mọi ưu và nhược của nó đều là hệ quả của câu đó.

1.6 Điều gì chứng minh cách hiểu này sai

Mệnh đề chính — *bộ lọc là MAP đã marginalize toàn bộ quá khứ và tuyến tính hoá một lần* — là một tương đương toán học dưới giả thiết Gauss và mô hình đúng (cửa a, đối chiếu [12] và [13], cửa c). Nó bị bác bỏ nếu ai đó chỉ ra một bộ lọc mà hành vi của nó *không* tái tạo được bằng một bài toán MAP với marginalization tương ứng. Các bộ lọc phi Gauss — particle filter — nằm ngoài phạm vi này, và đó là một giới hạn thật của khung ở đây chứ không phải một ngoại lệ nhỏ.

Mệnh đề thứ hai — *EKF-SLAM có chi phí bậc hai* — là kết quả đếm phép toán, kiểm được bằng mini project (cửa b). Nó bị bác bỏ nếu một cài đặt khai thác cấu trúc nào đó trong Σ để đạt chi phí thấp hơn — và thực tế có những công trình làm điều này bằng cách xấp xỉ Σ bằng một ma trận thưa, với cái giá là mất tính chính xác của hiệp phương sai.

Mệnh đề thứ ba — *dạng căn bậc hai giảm số điều kiện xuống căn bậc hai* — là một đồng nhất thức đại số. Điều *có thể* sai là kết luận thực dụng rằng điều đó đáng kể: với bài toán có số điều kiện nhỏ, lợi ích không đo được. Mini project ở trên không kiểm điều này; đó là một thí nghiệm riêng đáng làm, và nó nằm trong 99-chua-biet.md.

Một mệnh đề mềm hơn: chương này khẳng định *bốn tiêu chí ở khối Cam bẫy quyết định hơn độ chính xác trung bình trong sản phẩm*. Đó là một phán đoán về ưu tiên kỹ thuật, không phải một sự kiện đo được, và nó có thể sai trong ứng dụng mà độ chính xác là yêu cầu chính — chẳng hạn quét đo đạc, nơi độ trễ không quan trọng và chỉ độ chính xác cuối cùng mới tính. Nêu rõ điều này vì cây này viết với mục tiêu sản phẩm robot, và ưu tiên ấy không phổ quát (*tôi suy ra từ mục tiêu đã nêu ở 00-map, chưa đổi chiều nguồn*).

1.7 Kết nối

Nối lên trên. A/04-uoc-luong-map-va-bundle-adjustment là khung mà toàn bộ chương này nằm trong; đặc biệt mục 5 ở đó về marginalization là công cụ chính của mục 2 ở đây. A/05-observability-gauge-va-suy-bien mục 6 giải thích bệnh bất nhất mà FEJ chữa — không có chương đó thì FEJ trông như một thủ thuật kỳ quặc. A/06-bat-dinh-va-lan-truyen-sai-so giải thích vì sao Σ đặc còn Λ thưa, tức gốc của chi phí bậc hai ở mục 2B. B/08-mo-hinh-phan-du mục 3C cho structureless, kỹ thuật làm MSCKF khả thi.

Nối xuống dưới. B/10-khoi-tao là chỗ mọi kiến trúc đều cần một khởi đầu, và yêu cầu về chất lượng khởi đầu khác nhau giữa chúng — bộ lọc nhạy hơn vì nó tuyến tính hoá một lần. B/11-loop-closure-va-nhat-quan-toan-cuc xây trực tiếp trên pose graph ở mục 4, và nó là chương giải thích vì sao khả năng đóng vòng — thứ mà MSCKF không có — lại quyết định tính dùng được của một hệ trong không gian lặp lại. C/13-quan-ly-ban-do là chỗ kiến trúc nhiều luồng trở thành một vấn đề kỹ thuật thật: tracking, local mapping và loop closing chạy ở ba tần số khác nhau và phải chia nhau một bản đồ.

Nói xa hơn. C/15-hop-nhat-da-cam-bien cho thấy lựa chọn kiến trúc ở đây quyết định việc thêm cảm biến dễ hay khó: với đồ thị nhân tử thì thêm một loại nhân tử là đủ, còn với bộ lọc thì phải sửa cả vector trạng thái và các ma trận. D/20-toi-uu-hieu-nang-va-he-thong lấy phân vị thời gian — không phải trung bình — làm tiêu chí, và giải thích vì sao một hệ nhanh trung bình nhưng thỉnh thoảng chậm là không dùng được trong vòng điều khiển.

Nói ngang: ../IMU_Fusion/ chương 8 và 9 bàn cùng chủ đề với trọng tâm quán tính, và nói kỹ hơn về ESKF, InEKF và cấu trúc nhân tử tiền tích phân. Chỗ nào cần chi tiết về bộ lọc bất biến thì đọc ở đó; chương này cố ý dừng ở mức nêu nó tồn tại và nó chứa cái gì.

Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Nêu hai trục phân biệt mọi kiến trúc back-end, và đặt EKF, MSCKF, VINS-Mono và iSAM2 lên hai trục đó.
2. Vì sao Λ thừa mà Σ đặc? Trả lời bằng một câu về ý nghĩa vật lý, không bằng đại số.
3. MSCKF rẻ và chính xác nhưng không phải SLAM. Vì sao, và hậu quả gì cho một robot chạy tám tiếng trong một nhà kho?
4. iSAM2 giữ toàn bộ lịch sử mà vẫn chạy thời gian thực. Loại cập nhật nào làm nó đắt, và vì sao?
5. Tự dẫn lại vì sao $\text{cond}(\Lambda) = \text{cond}(R)^2$, rồi nói điều đó nghĩa là mất bao nhiêu chữ số với bài toán có $\text{cond}(\Lambda) = 10^{10}$.
6. Pose graph nén thông tin một cách không phục hồi được. Cụ thể cái gì bị mất, và nó liên hệ thế nào với bệnh quá tự tin ở A/06?
7. Nêu bốn tiêu chí quyết định lựa chọn back-end trong sản phẩm, và nói vì sao độ chính xác trung bình không nằm trong bốn cái đó.

Nguồn

- [1] G. Klein and D. Murray, “Parallel Tracking and Mapping for Small AR Workspaces,” in *IEEE and ACM International Symposium on Mixed and Augmented Reality (ISMAR)*, mức tin A; công trình tách tracking khỏi mapping thành hai luồng, 2007.
- [2] A. I. Mourikis and S. I. Roumeliotis, “A Multi-State Constraint Kalman Filter for Vision-Aided Inertial Navigation,” in *IEEE International Conference on Robotics and Automation (ICRA)*, mức tin A; nguồn của cách khử landmark bằng chiếu lên null space, 2007.
- [3] M. Kaess, H. Johannsson, R. Roberts, et al., “iSAM2: Incremental Smoothing and Mapping Using the Bayes Tree,” *The International Journal of Robotics Research*, vol. 31, no. 2, pp. 216–235, 2012, mức tin A.
- [4] H. Strasdat, J. M. M. Montiel, and A. J. Davison, “Scale Drift-Aware Large Scale Monocular SLAM,” in *Robotics: Science and Systems (RSS)*, mức tin A; nguồn của loop closure trên $Sim(3)$, 2010.
- [5] V. Usenko, N. Demmel, D. Schubert, et al., “Visual-Inertial Mapping with Non-Linear Factor Recovery,” *IEEE Robotics and Automation Letters*, vol. 5, no. 2, pp. 422–429, 2020, mức tin B; hệ thống Basalt.
- [6] N. Demmel, C. Sommer, D. Cremers, et al., “Square Root Bundle Adjustment for Large-Scale Reconstruction,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin B, 2021.
- [7] D. M. Rosen, L. Carlone, A. S. Bandeira, et al., “SE-Sync: A Certifiably Correct Algorithm for Synchronization over the Special Euclidean Group,” *The International Journal of Robotics Research*, vol. 38, no. 2-3, pp. 95–125, 2019, mức tin A.
- [8] R. Hartley, J. Trumpf, Y. Dai, et al., “Rotation Averaging,” *International Journal of Computer Vision*, vol. 103, no. 3, pp. 267–305, 2013, mức tin A.
- [9] P. Geneva, K. Eickenhoff, W. Lee, et al., “OpenVINS: A Research Platform for Visual-Inertial Estimation,” in *IEEE International Conference on Robotics and Automation (ICRA)*, mức tin A, 2020.
- [10] T. Qin, P. Li, and S. Shen, “VINS-Mono: A Robust and Versatile Monocular Visual-Inertial State Estimator,” *IEEE Transactions on Robotics*, vol. 34, no. 4, pp. 1004–1020, 2018, mức tin A.
- [11] C. Campos, R. Elvira, J. J. Gómez Rodríguez, et al., “ORB-SLAM3: An Accurate Open-Source Library for Visual, Visual-Inertial, and Multimap SLAM,” *IEEE Transactions on Robotics*, vol. 37, no. 6, pp. 1874–1890, 2021, mức tin A.
- [12] T. D. Barfoot, *State Estimation for Robotics*. Cambridge University Press, 2017, mức tin A; nguồn chuẩn cho ước lượng trên đa tạp.
- [13] S. Thrun, W. Burgard, and D. Fox, *Probabilistic Robotics*. MIT Press, 2005, mức tin A.